

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PARÁ

CAMPUS MARABÁ INDUSTRIAL

CURSO: Licenciatura em Informática

TURNO: Noturno

DISCIPLINA: Tópicos em Segurança da Informação

PROFESSOR: Diogo Alvino

Projeto - Monitor de Arquivos Seguros

Desenvolver um monitor simples de integridade de arquivos em Python que detecta criação, alteração e exclusão em uma pasta específica.

O projeto busca consolidar conceitos de integridade, auditoria, análise de riscos e segurança lógica, cobrindo parte significativa da ementa da disciplina.

Estrutura Base

Cada aluno:

1. Monitorar a pasta **/DocumentosSeguros** e detectar criação / alteração / exclusão;
2. Gerar hash SHA-256 de cada arquivo e comparar com o baseline (hashes.json);
3. Registrar logs (**logs_monitor.log**) com data, hora e tipo de evento;
4. Gerar relatório final (**relatorio.txt ou .md**) com resumo dos eventos e uma análise simples de risco (ex: quais arquivos foram mais modificados, possíveis causas e medidas preventivas).

Entregáveis

Cada aluno deverá entregar:

- Arquivo principal monitor.py
- Arquivos gerados: hashes.json, logs_monitor.log, relatorio.txt
- Um pequeno README (máx. 1 página) explicando:
 - como executar o código;
 - qual variação individual foi implementada;

- e uma print do terminal mostrando seu recurso funcionando.

Variações sorteadas (1 item adicional por aluno)

- **CLAUDEBERGUISON SILVA PEREIRA**

- **Backup simples**
- A cada varredura, criar uma cópia do hashes.json em backups/ hashes_YYYYMMDD.json. Garante a recuperação em caso de falha.

- **MARCOS VINICIUS PEREIRA OLIVEIRA**

- **Contador de eventos**
- No relatório final, apresentar o total de arquivos criados, alterados e excluídos durante a execução. Ex: "Total criados: 3 / alterados: 5 / excluídos: 1".

- **MURILO VINICIUS MORAES DA SILVA**

- **Log por nível**
- Classificar os eventos no log: INFO (criação), WARNING (alteração) e ALERT (exclusão). Exemplo: `2025-10-09 19:40`

- **RODRIGO PEREIRA DOS SANTOS**

- **Registro de usuário**
- Cada linha do log deve incluir o usuário do sistema que executou o monitor (via getpass.getuser()). Ex: `user=rodrigo`

- **THIAGO ALVES DA COSTA**

- **Deteção de duplicatas**
- O programa deve identificar arquivos diferentes com o mesmo hash e registrar no log: "POSSÍVEL DUPLICATA: arquivo1.txt e arquivo2.txt".

Etapa 1 - 09/10 - Planejamento do Projeto

Objetivo: Apresentar a proposta do projeto, revisar os conceitos de integridade, riscos e incidentes de segurança, e definir a estrutura básica do código e das pastas. Cada aluno deve identificar seu tema individual (variação) e planejar como implementará.

Entregáveis: Esboço ou planejamento individual (no caderno ou digital) contendo nome do aluno, variação sorteada, descrição breve do funcionamento esperado e confirmação da criação da pasta /DocumentosSeguros no computador.

Etapa 2 - 16/10 - Implementação do Baseline e Hash

Objetivo: Criar o script inicial que percorre a pasta e gera o arquivo hashes.json com o hash SHA-256 de cada arquivo. Essa versão cria o “baseline” de integridade.

Entregáveis: Arquivo monitor.py (versão 1) com a função de varredura e geração do hashes.json.

Etapa 3 - 23/10 - Detecção e Logs

Objetivo: Implementar a parte que detecta criação, alteração e exclusão de arquivos comparando com o baseline e registrando tudo no arquivo logs_monitor.log.

Entregáveis: Arquivo monitor.py (versão 2) funcional, gerando logs com data, hora e tipo de evento; arquivo logs_monitor.log comprovando a detecção dos eventos.

Etapa 4 - 30/10 - Variação Individual

Objetivo: Incluir no código o recurso individual sorteado (backup, contador, níveis de log, registro de usuário ou duplicatas) e garantir que o recurso funcione corretamente em conjunto com o monitor.

Entregáveis: Arquivo monitor.py (versão 3) com a variação implementada e testada via terminal.

Etapa 5 - 06/11 - Relatório Final

Objetivo: Gerar o relatório relatorio.txt ou relatorio.md contendo um resumo dos eventos detectados e uma breve análise de risco, indicando quais arquivos sofreram mais alterações e quais medidas podem reduzir esses incidentes.

Entregáveis: Relatório relatorio.txt (ou .md) completo com resumo e análise de risco; código final atualizado.

Etapas 6 - 13/11 - Revisão e Ajustes

Objetivo: Revisar o código e corrigir possíveis erros. Testar novamente o funcionamento do monitor, dos logs e da geração do relatório. Verificar se todos os arquivos são criados corretamente e se não há falhas de execução.

Entregáveis: Versão final do monitor.py testada e funcionando, sem erros; relatório e logs revisados.

Etapas 7 - 20/11 - Entrega Técnica

Objetivo: Entregar oficialmente o projeto completo ao professor, incluindo todos os arquivos do sistema e documentação.

Entregáveis:

- monitor.py
- hashes.json
- logs_monitor.log
- relatorio.txt
- README (1 página explicando o funcionamento e a variação individual)
- Print do terminal mostrando o recurso individual em execução.